SECCIÓN 3: LAS BASES DE PROJECT REACTOR

11. Introducción a Project Reactor

```
Project Reactor es una librería basada en el
paradigma de programación reactiva para la
JVM. Está diseñada para manejar flujos de
datos asíncronos y no bloqueantes de manera
eficiente, utilizando el modelo de
programación basado en Reactive Streams.
```

➔¿Dónde se usa?

```
• Bases de datos reactivas (ej. R2DBC para bases de datos SQL reactivas,
  MongoDB Reactive)
• Clientes HTTP asíncronos (ej. WebClient en Spring)
• Procesamiento de eventos y streaming de datos (ej. Kafka, RabbitMQ)
• Aplicaciones de alto rendimiento y baja latencia, como microservicios y
  sistemas de mensajería en tiempo real.
```

➔Ventajas:

```
✅ Escalabilidad: Maneja un gran volumen de
peticiones concurrentes sin bloquear hilos.
✅ Eficiencia en el uso de recursos: Reduce
el uso excesivo de hilos gracias al modelo
de backpressure.
✅ Composición funcional: Facilita la
manipulación de flujos de datos con
operadores.
✅ Integración con Spring: Se usa en Spring
WebFlux, Spring Cloud Gateway, Spring
Security Reactive, entre otros.
```

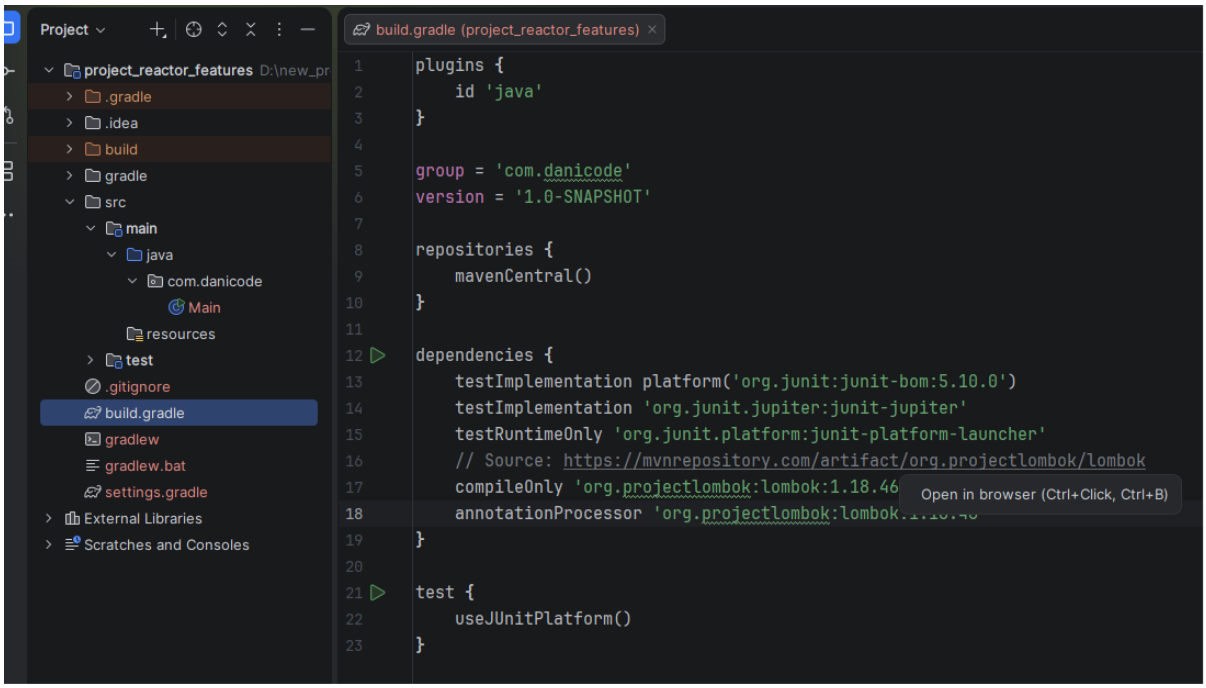
➔Desventajas:

```
❌ Curva de aprendizaje pronunciada:
No es trivial para desarrolladores
acostumbrados a la programación
imperativa.
❌ Depuración más compleja: Las
excepciones pueden ser difíciles de
rastrear debido a la naturaleza
asíncrona del código.
❌ No siempre necesario: En
aplicaciones tradicionales basadas
en JDBC o donde la latencia no es un
problema crítico, el modelo
bloqueante de Spring MVC sigue
siendo más fácil de manejar.
```

Mono	Flux
Representa un único valor o un vacío (empty). Se usa para operaciones que devuelven 0 o 1 resultado.	Representa una secuencia de múltiples valores (0..N). Se usa para streams de datos o colecciones asíncronas.
Ejemplo: Un único usuario buscado por ID.	*Ejemplo: Un listado de usuarios en una base de datos.*
Equvalente a Optional<Y>	Equivalente a List<T>

12. Creando un proyecto con Java y Project Reactor

➔Agregando dependencia lombok

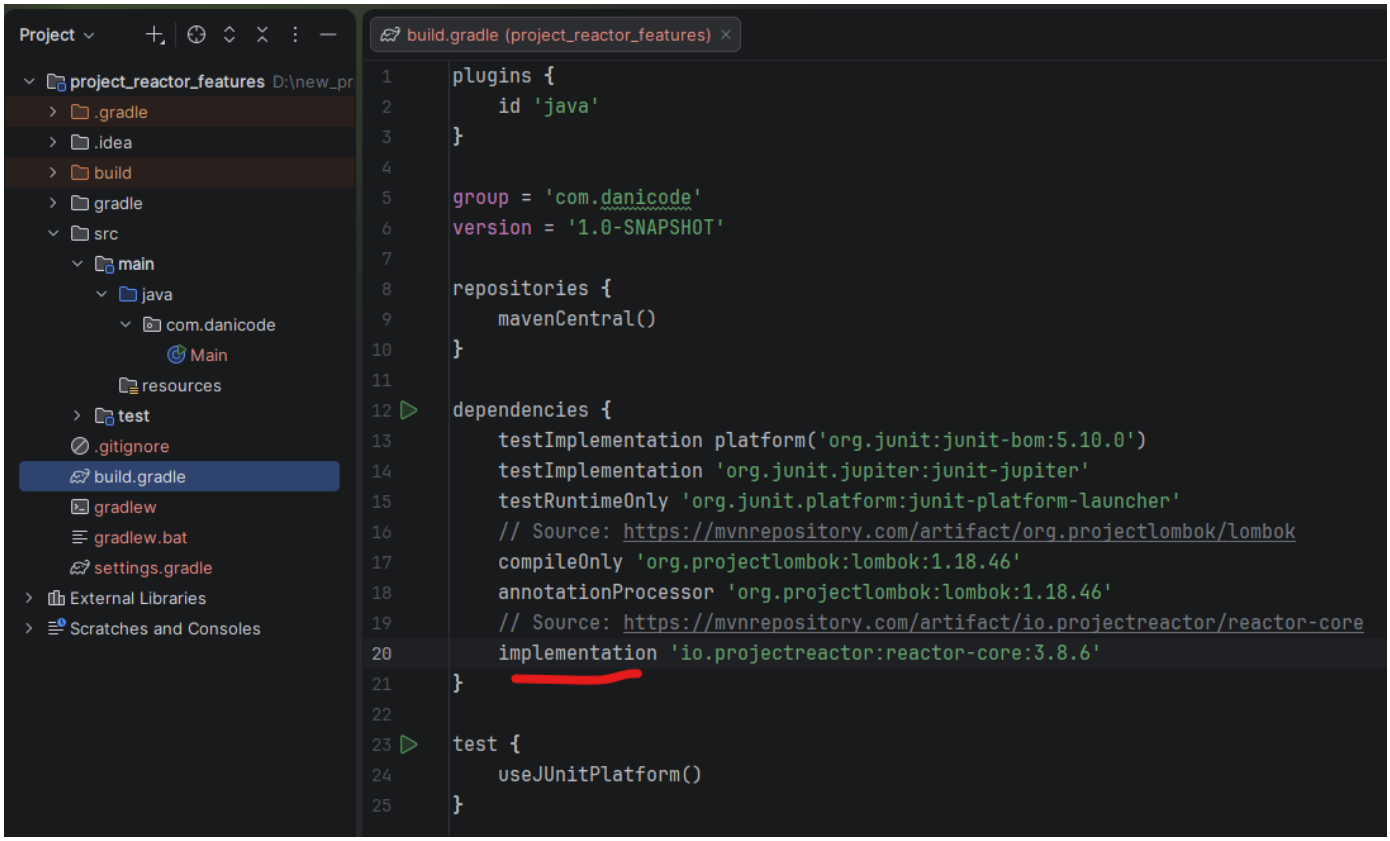


➔Agregando dependencia de projectreactor

https://projectreactor.io/docs

// Source: https://mvnrepository.com/artifact/io.projectreactor/reactor-core

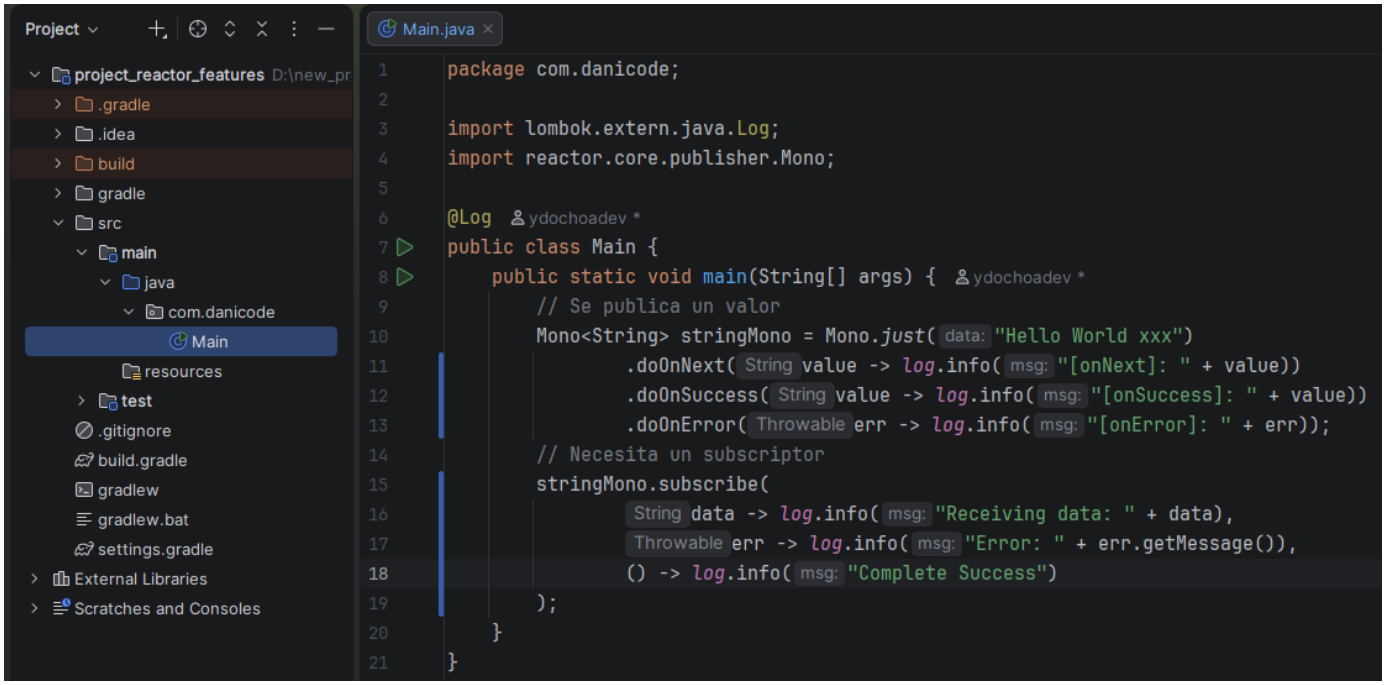
implementation 'io.projectreactor:reactor-core:3.8.6'



➔Probando que funcione la dependencia:



13. ¿Qué es un Mono?



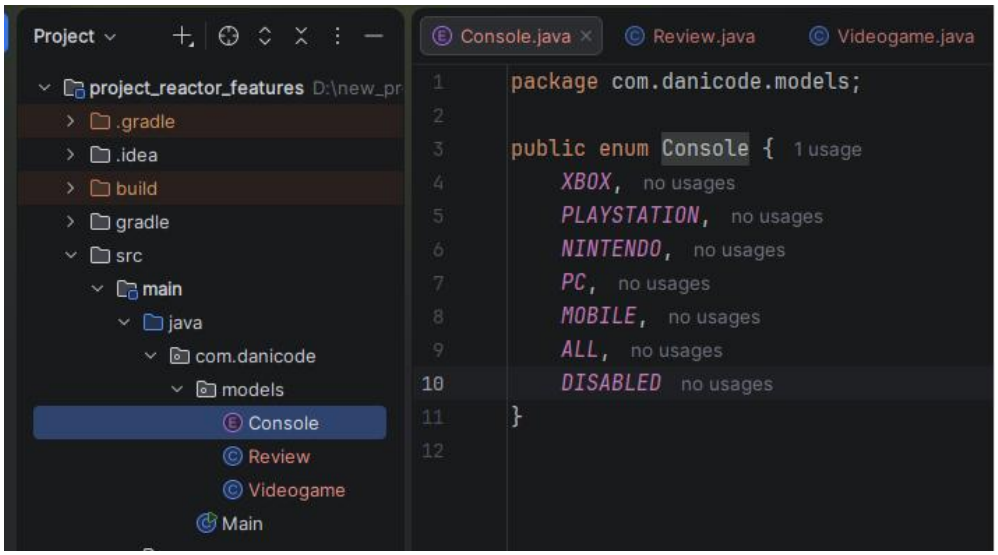
14. ¿Qué es un Flux?

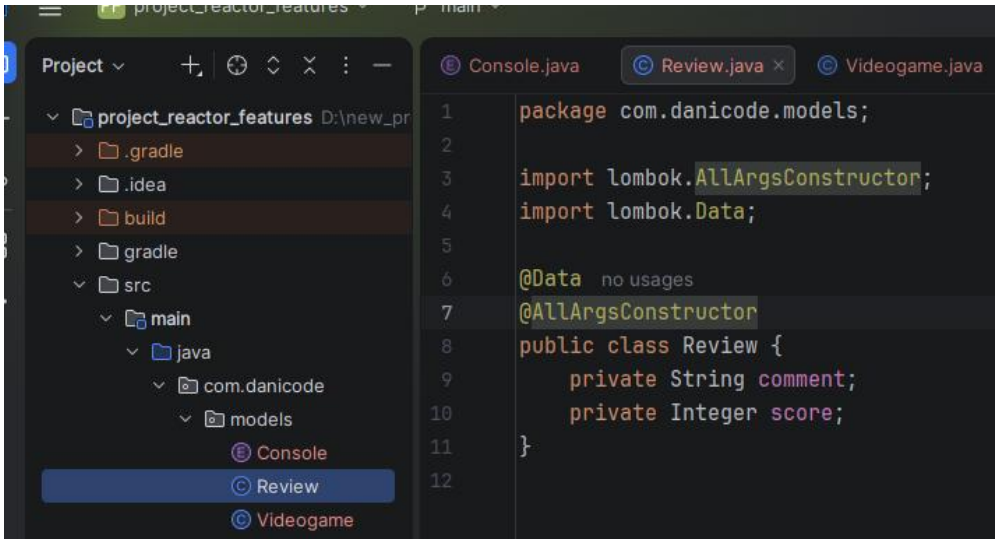
```java
@Log ydochoadev *
public class Main {
    public static void main(String[] args) { ydochoadev *
        // Se publica un valor
        Mono<String> stringMono = Mono.just( data: "Hello World xxx")
                .doOnNext( String value -> log.info( msg: "[onNext]: " + value))
                .doOnSuccess( String value -> log.info( msg: "[onSuccess]: " + value))
                .doOnError( Throwable err -> log.info( msg: "[onError]: " + err));
        // Necesita un subscriptor
        stringMono.subscribe(
                String data -> log.info( msg: "Receiving data: " + data),
                Throwable err -> log.info( msg: "Error: " + err.getMessage()),
                () -> log.info( msg: "Complete Success")
        );

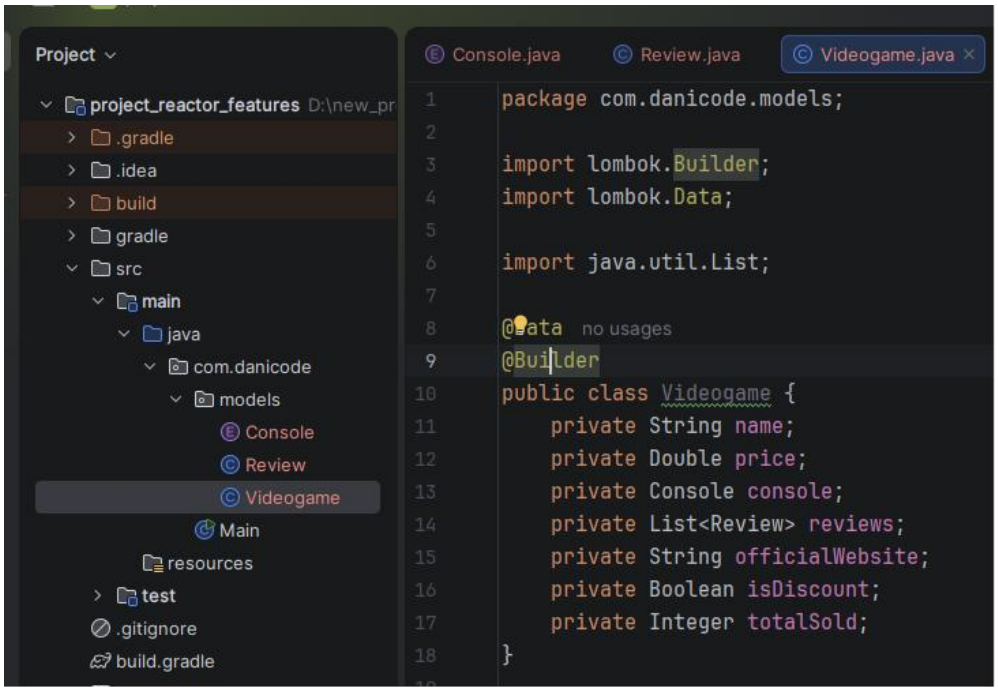
        // Publicador Flux
        // doOnNext => Se ejecuta varias veces
        // doOnComplete => Solo se ejecuta 1 vez
        Flux<String> stringFlux = Flux.just( ...data: "Java", "Spring", "Reactor")
                .doOnNext( String data -> log.info( msg: "[onNext]: " + data))
                .doOnComplete(() -> log.info( msg: "[OnComplete]: Success"));
        // Consumidor del Flux
        stringFlux.subscribe(
                String data -> log.info( msg: "Received data: " + data),
                Throwable err -> log.info( msg: "Error: " + err.getMessage()),
                () -> log.info( msg: "Complete Success")
        );
    }
}
```

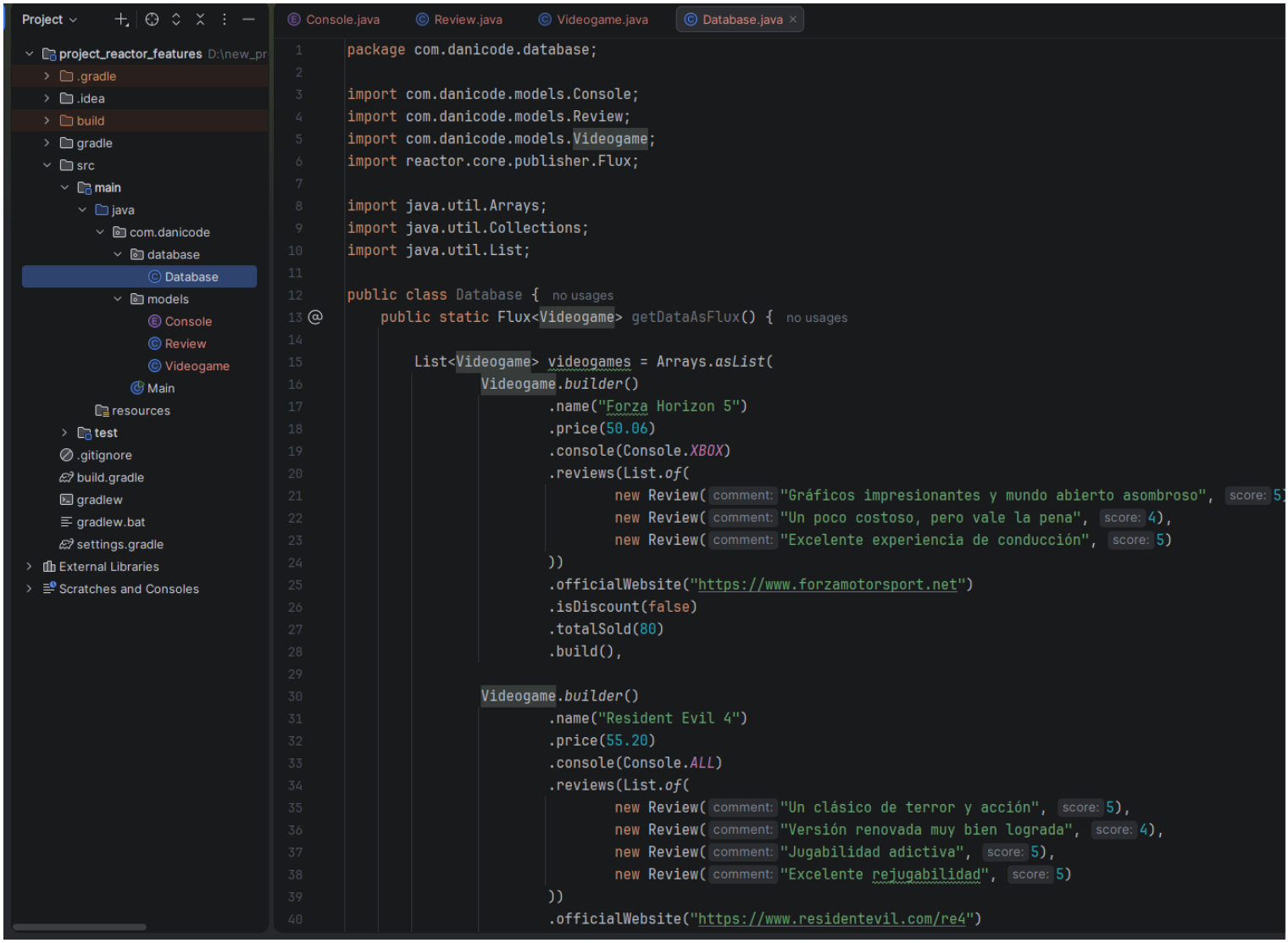
15. Creando un Flux con objetos complejos

➔Creando clases:

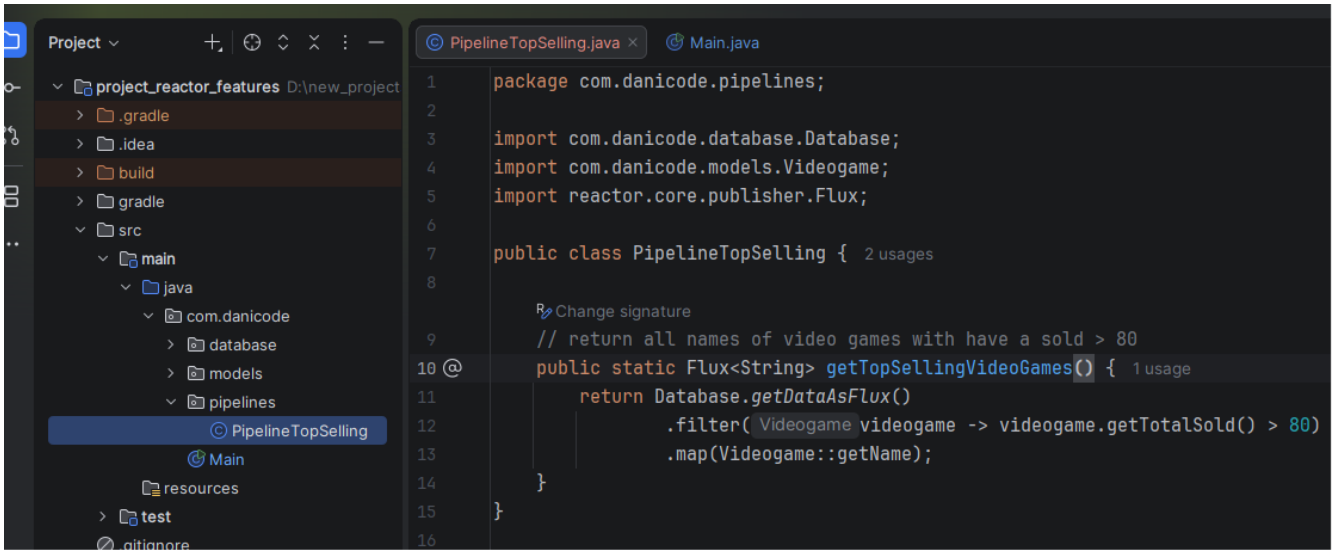








16. Trabajando con Pipelines: map, filter y reduce



➔ En el main:

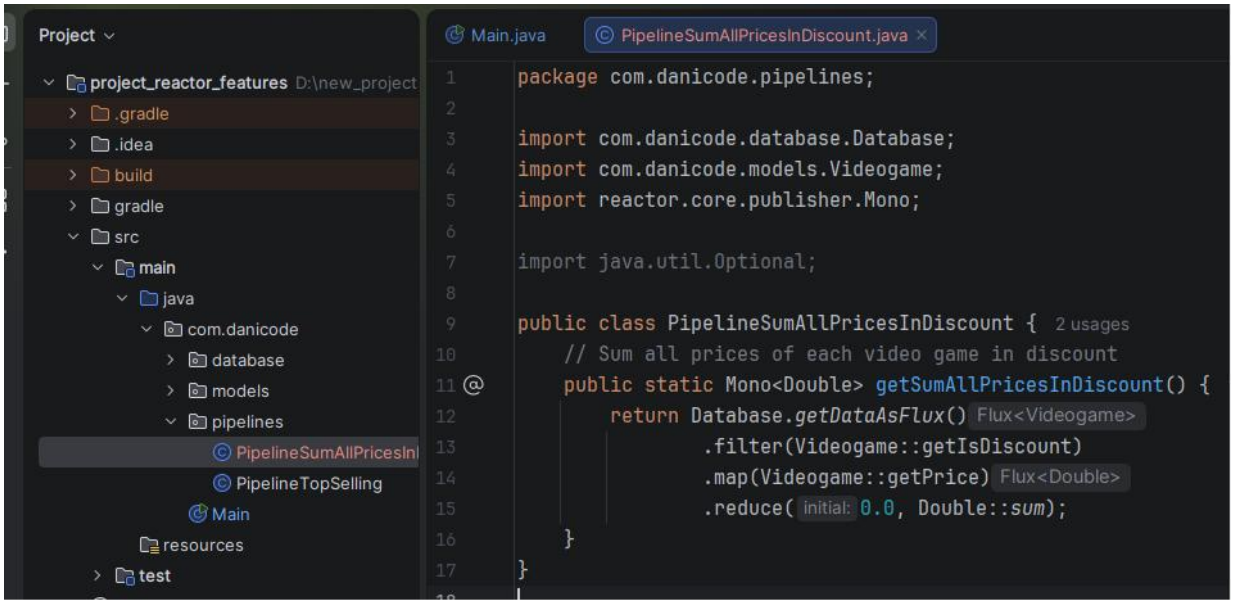
```java
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

@Log  ydochoadev *
public class Main {
    public static void main(String[] args) {  ydochoadev *
        // Se publica un valor
        /* Mono<String> stringMono = Mono.just("Hello World xxx")
                .doOnNext(value -> log.info("[onNext]: " + value))
                .doOnSuccess(value -> log.info("[onSuccess]: " + value))
                .doOnError(err -> log.info("[onError]: " + err));
        // Necesita un subscriptor
        stringMono.subscribe(
                data -> log.info("Receiving data: " + data),
                err -> log.info("Error: " + err.getMessage()),
                () -> log.info("Complete Success")
        );

        // Publicador Flux
        // doOnNext => Se ejecuta varias veces
        // doOnComplete => Solo se ejecuta 1 vez
        Flux<String> stringFlux = Flux.just("Java", "Spring", "Reactor")
                .doOnNext(data -> log.info("[onNext]: " + data))
                .doOnComplete(() -> log.info("[OnComplete]: Success"));
        // Consumidor del Flux
        stringFlux.subscribe(
                data -> log.info("Received data: " + data),
                err -> log.info("Error: " + err.getMessage()),
                () -> log.info("Complete Success")
        );*/

        PipelineTopSelling.getTopSellingVideoGames()
                .subscribe(System.out::println);
    }
}
```

17. Ejercicio con Pipelines



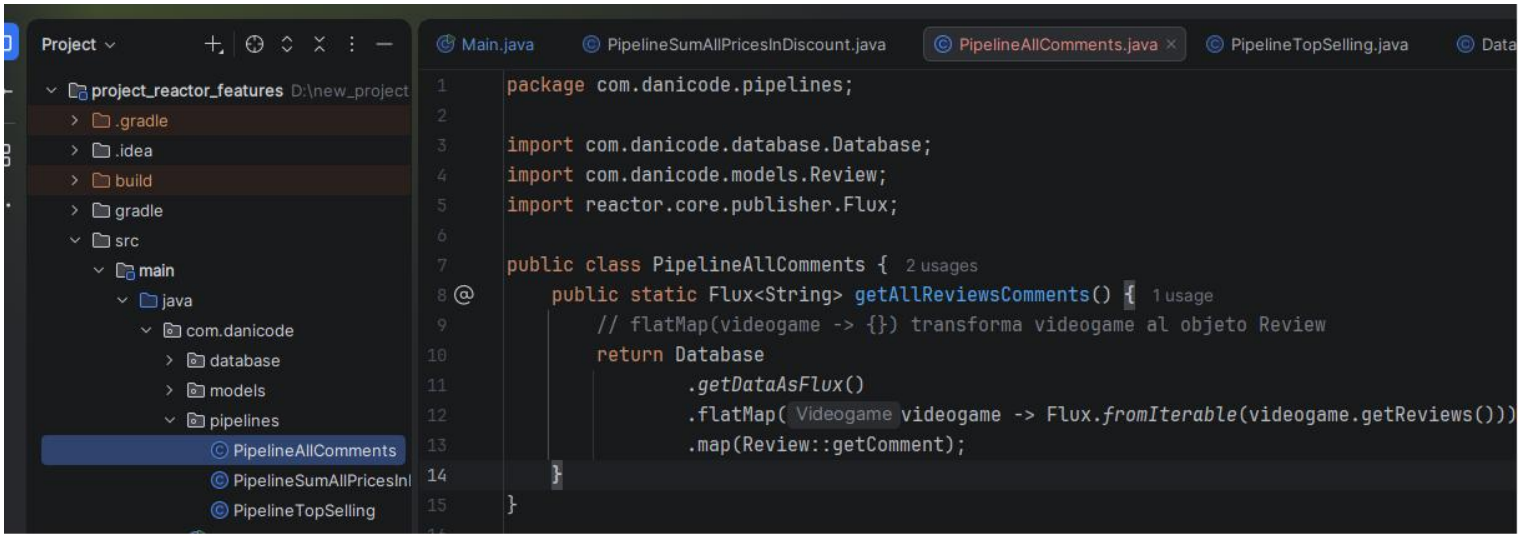
Main

```java
        PipelineSumAllPricesInDiscount.getSumAllPricesInDiscount()
                .subscribe(System.out::println);
    }
```

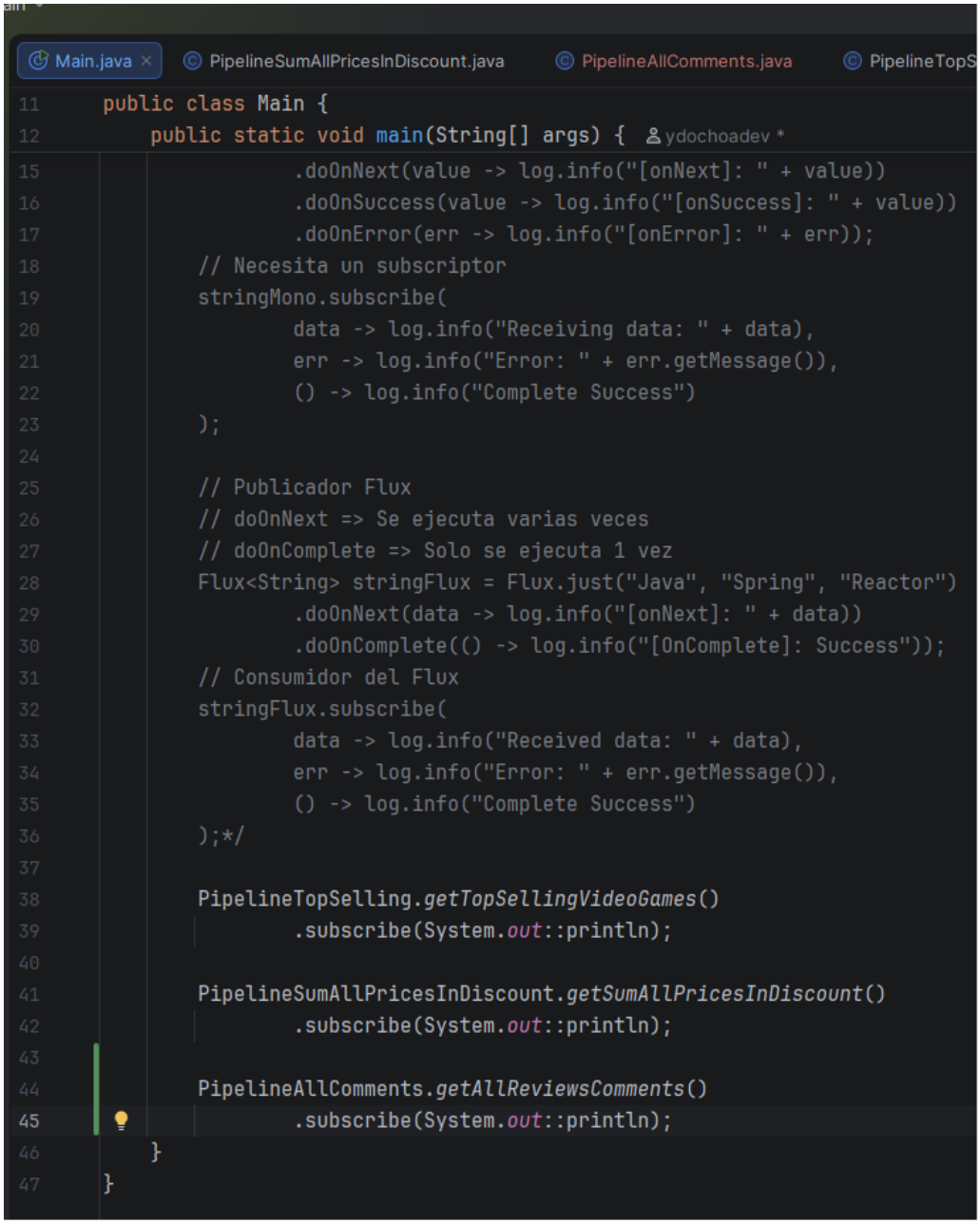
18. Operador flatMap

Se usa flatMap para combinar 2 flujos.

➔Se crea otro pipeline:



➔Suscribirse al publicador:



19. Convirtiendo dos flujos con flatMap

```
                .subscribe(System.out::println);

        PipelineAllComments.getAllReviewsComments()
                .subscribe(System.out::println);*/

        // Combinación de 2 flujo con flapMap
        Flux<String> fluxA = Flux.just( ...data: "1", "2");
        Flux<String> fluxB = Flux.just( ...data: "A", "B");
        // Tercer flux combinado
        Flux<String> combinedFlux = fluxA.flatMap( String strA -> fluxB.map( String strB -> strA + " - " + strB));
        // Subscribirse
        combinedFlux.subscribe(System.out::println);
        combinedFlux
                .map(String::toLowerCase)
                .doOnNext(System.out::println)
                .subscribe();

    }
}
```

20. Operadores concat y merge

➔merge: Junta los flujos por orden de llegada

```
Main.java
public class Main {
    public static void main(String[] args) {  ydochoadev *
                .doOnNext(System.out::println)
                .subscribe();*/

        // Merge
        log.info( msg: "****** MERGE ******");
        Flux<String> combineFlux2 = Flux.merge(fluxA, fluxB);
        combineFlux2
                .doOnNext(System.out::println)
                .blockLast(); // Evita terminar la ejecución antes de procesar los flujos

    }
}
```

```
code.Main.main()]

Main.mair 1 sec, 37 ms
20:23:46: Executing ':com.danicode.Main.main()'…

> Task :compileJava
> Task :processResources NO-SOURCE
> Task :classes

> Task :com.danicode.Main.main()
A
1
B
2
3
```

➔concat: Concatena el primer flujo con el segundo, de manera ordenada.

```
Main.java
9   public class Main {
10      public static void main(String[] args) {  ydochoadev *
56                  .doOnNext(System.out::println)
57                  .subscribe();*/
58
59          // Merge
60          log.info( msg: "****** CONCAT ******");
61          Flux<String> combineFlux2 = Flux.concat(fluxA, fluxB);
62          combineFlux2
63                  .doOnNext(System.out::println)
64                  .blockLast(); // Evita terminar la ejecución antes de procesar los flujos
65
66      }
67  }

icode.Main.main()]

.Main.ma 1 sec, 130 ms
20:24:27: Executing ':com.danicode.Main.main()'…

> Task :compileJava
> Task :processResources NO-SOURCE
> Task :classes

> Task :com.danicode.Main.main()
1
2
3
A
B
```

* Similando llamadas a 4 micros: Como ejemplo, habrá 4 flux simulando la respuesta de webClient.

```
log.info( msg: "****** OPERADOR ZIP ******");
// Envíos
Flux<String> fluxShipments = Flux.just( ...data: "Shipment 1", "Shipment 2", "Shipment 3").delayElements(Duration.ofMillis(120));
// Almacén
Flux<String> fluxWarehouse = Flux.just( ...data: "Stock 1", "Stock 2", "Stock 3").delayElements(Duration.ofMillis(50));
Flux<String> fluxPayments = Flux.just( ...data: "Payment 1", "Payment 2", "Payment 3").delayElements(Duration.ofMillis(150));
Flux<String> fluxConfirms = Flux.just( ...data: "Confirm 1", "Confirm 2", "Confirm  3").delayElements(Duration.ofMillis(20));
```

➔Se debe mantener el orden, Shipment 1 con Stock1, Payment 1 y Confirm 1: Como si fuese una BD

➔Zip es como una tupla



➔Zip ofrece un constructor por defecto que sirve para combinar 2 Flujos:

```
log.info( msg: "****** OPERADOR ZIP ******");
// Envíos
Flux<String> fluxShipments = Flux.just( ...data: "Shipment 1", "Shipment 2", "Shipment 3").delayElements(Duration.ofMillis(120));
// Almacén
Flux<String> fluxWarehouse = Flux.just( ...data: "Stock 1", "Stock 2", "Stock 3").delayElements(Duration.ofMillis(50));
Flux<String> fluxPayments = Flux.just( ...data: "Payment 1", "Payment 2", "Payment 3").delayElements(Duration.ofMillis(150));
Flux<String> fluxConfirms = Flux.just( ...data: "Confirm 1", "Confirm 2", "Confirm  3").delayElements(Duration.ofMillis(20));

Flux<String> reportFlux = Flux.zip(fluxShipments, fluxWarehouse, ( String shipment,  String stock) -> shipment + " - " + stock);
reportFlux.doOnNext(System.out::println).blockLast();
```

```
> Task :classes

> Task :com.danicode.Main.main()
1
2
3
A
B
Shipment 1 - Stock 1
Shipment 2 - Stock 2
Shipment 3 - Stock 3
```

➔Combinar varios flujos, lista

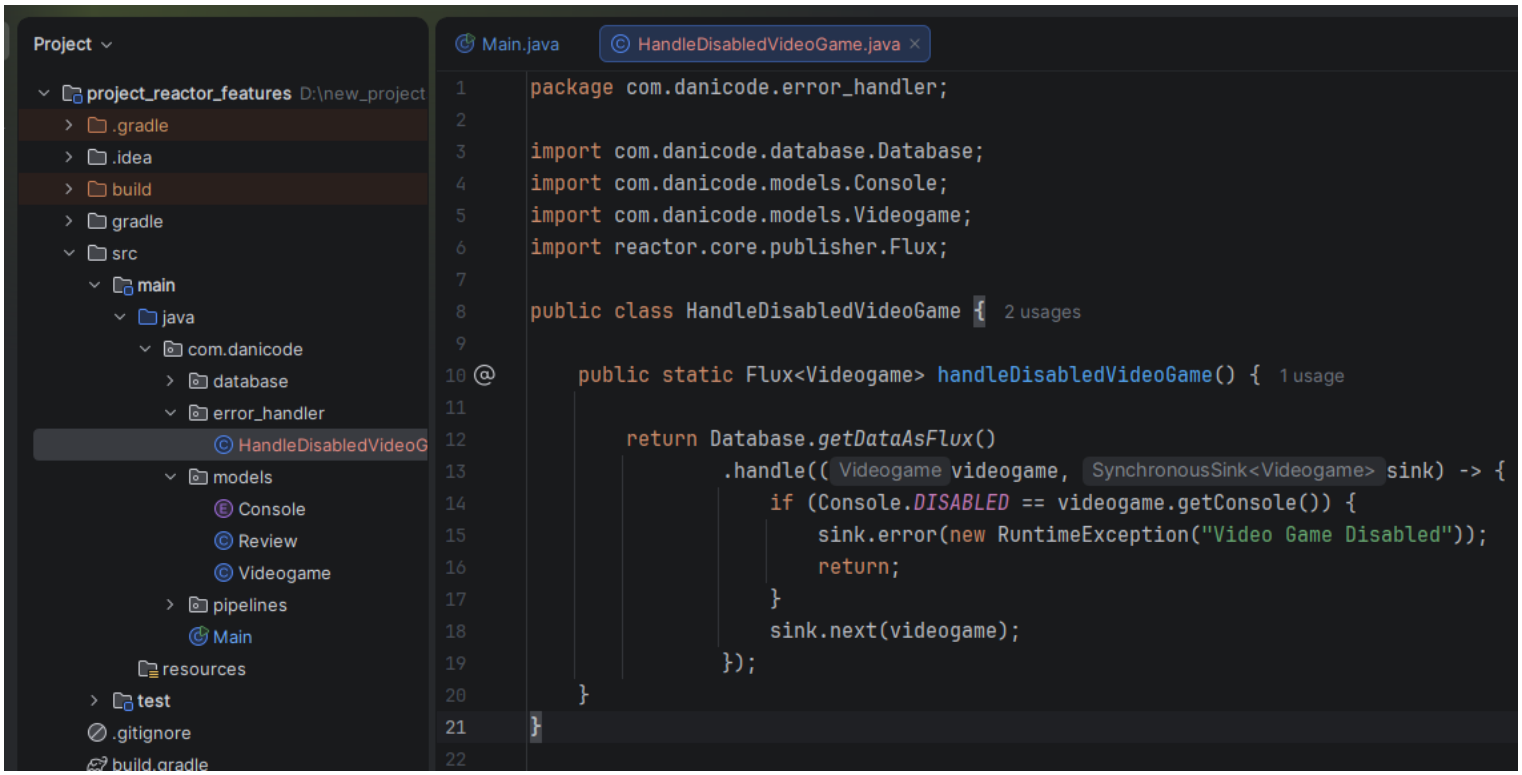
```
public static void main(String[] args) {  ydochoadev *
    log.info( msg: "****** OPERADOR ZIP ******");
    // Envíos
    Flux<String> fluxShipments = Flux.just( ...data: "Shipment 1", "Shipment 2", "Shipment 3").delayElements(Duration.ofMillis(120));
    // Almacén
    Flux<String> fluxWarehouse = Flux.just( ...data: "Stock 1", "Stock 2", "Stock 3").delayElements(Duration.ofMillis(50));
    Flux<String> fluxPayments = Flux.just( ...data: "Payment 1", "Payment 2", "Payment 3").delayElements(Duration.ofMillis(150));
    Flux<String> fluxConfirms = Flux.just( ...data: "Confirm 1", "Confirm 2", "Confirm  3").delayElements(Duration.ofMillis(20));

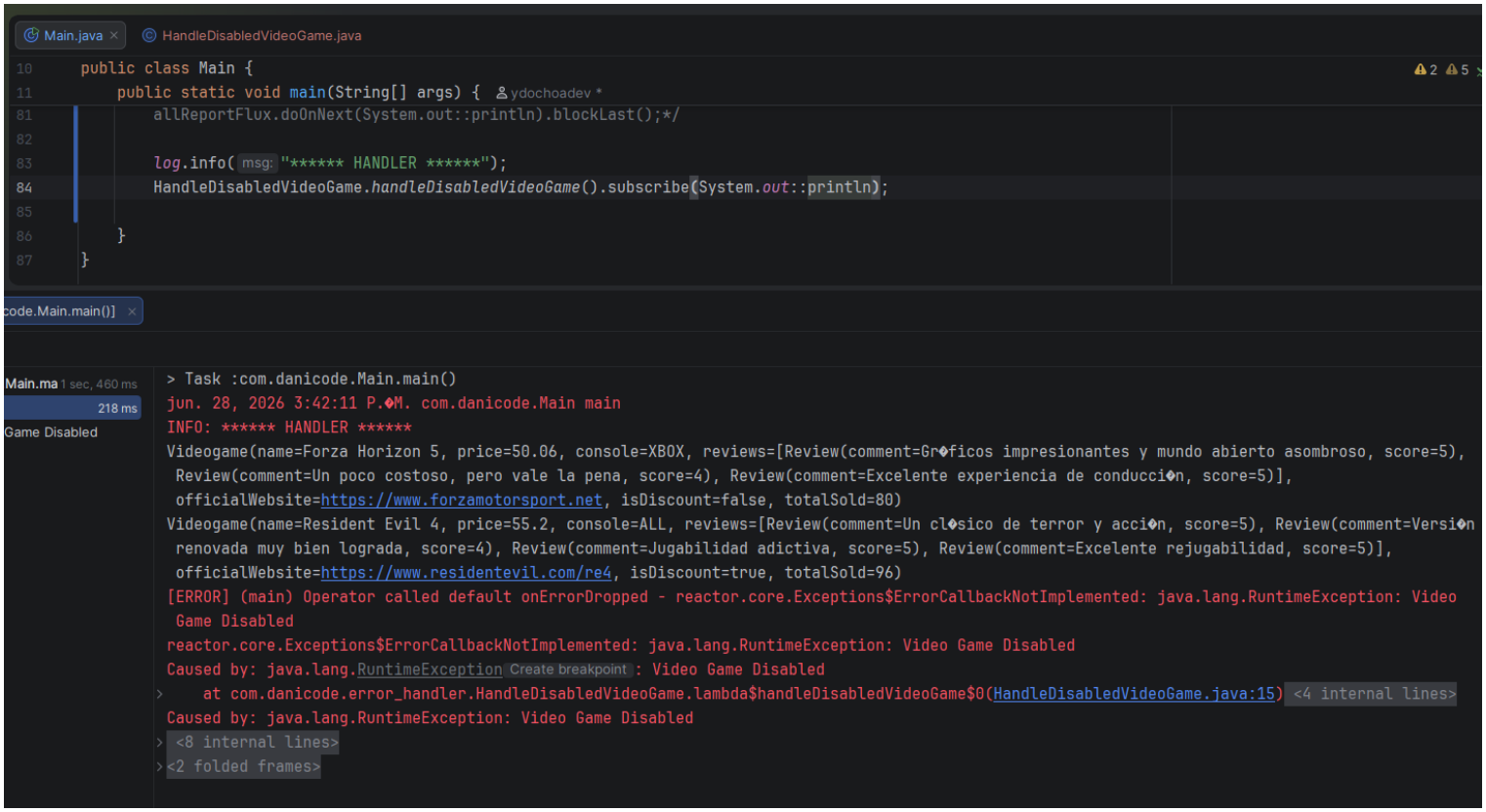
    // Combina 2 flujos
    // Flux<String> reportFlux = Flux.zip(fluxShipments, fluxWarehouse, (shipment, stock) -> shipment + " - " + stock);
    // Combina 2 varios flujos
    // Flx.zip(tupla1, tupla2,tupla3, tupla4)
    Flux<String> allReportFlux = Flux.zip(fluxShipments, fluxWarehouse, fluxPayments, fluxConfirms)
            .map( Tuple4<String, String, String, String> tuple -> tuple.getT1() + " - " + tuple.getT2() + " - " + tuple.getT3() + " - " + tuple.getT4());
    allReportFlux.doOnNext(System.out::println).blockLast();
```

```
> Task :com.danicode.Main.main()
1
2
3
A
B
Shipment 1 - Stock 1 - Payment 1 - Confirm 1
Shipment 2 - Stock 2 - Payment 2 - Confirm 2
Shipment 3 - Stock 3 - Payment 3 - Confirm  3
```

22. Operador handle

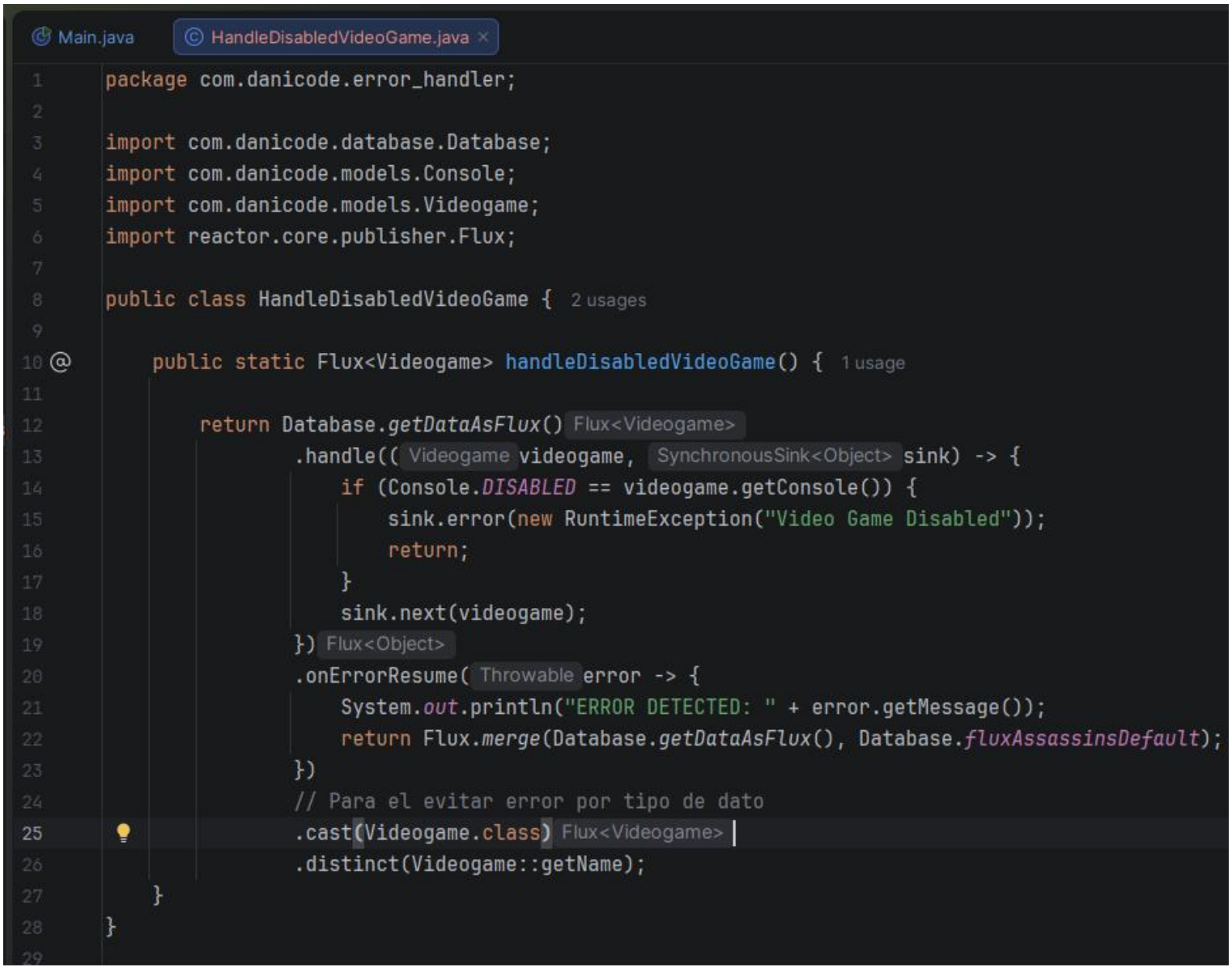
➔Similar al operador “filter” y mediante condiciones, puede lanzar errores.





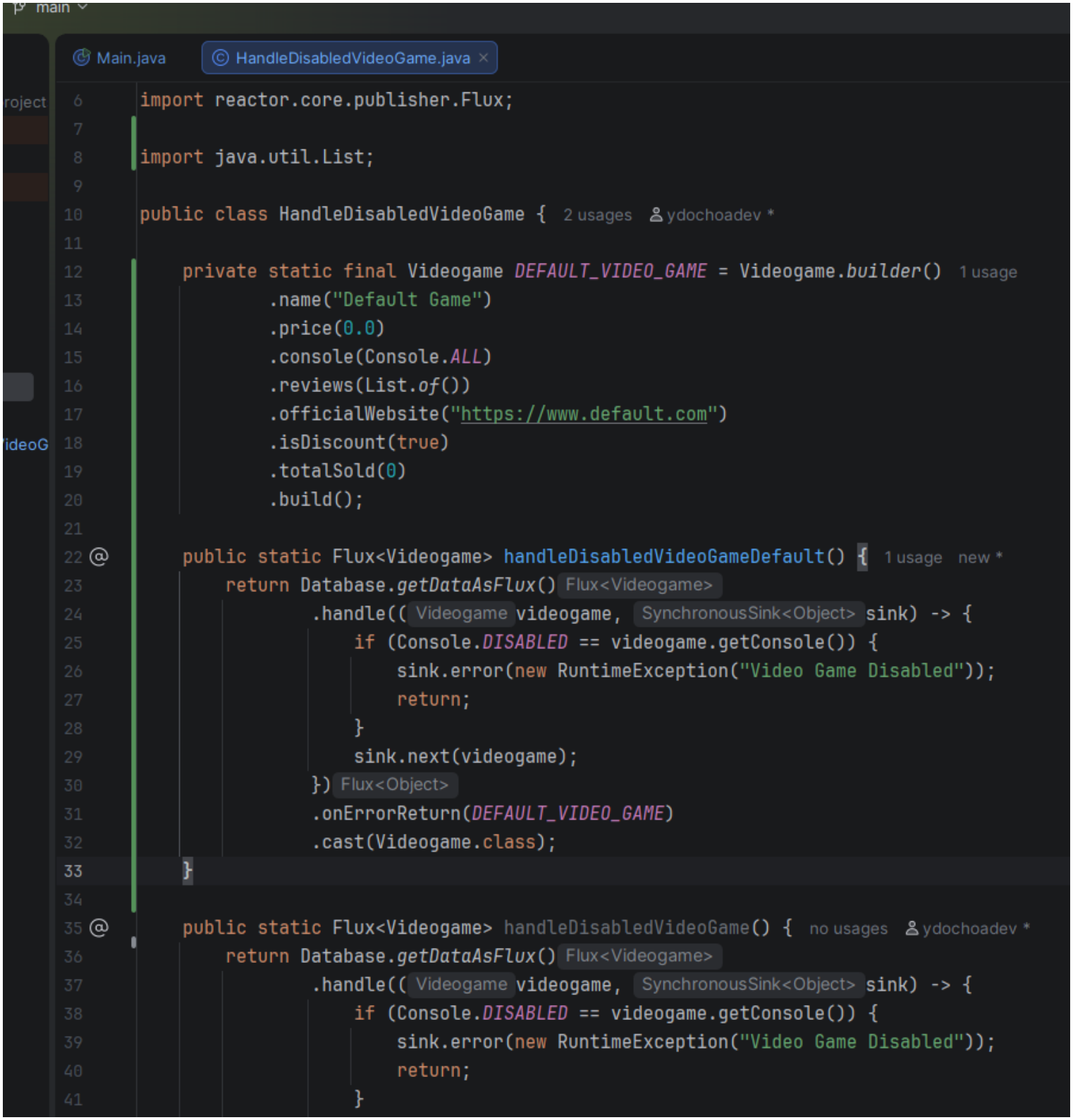
23. Manejo de errores con onErrorResume

➔Si hay una excepeción, se puede manejar emitiendo un nuvo flujo



24. Manejo de errores con onErrorReturn

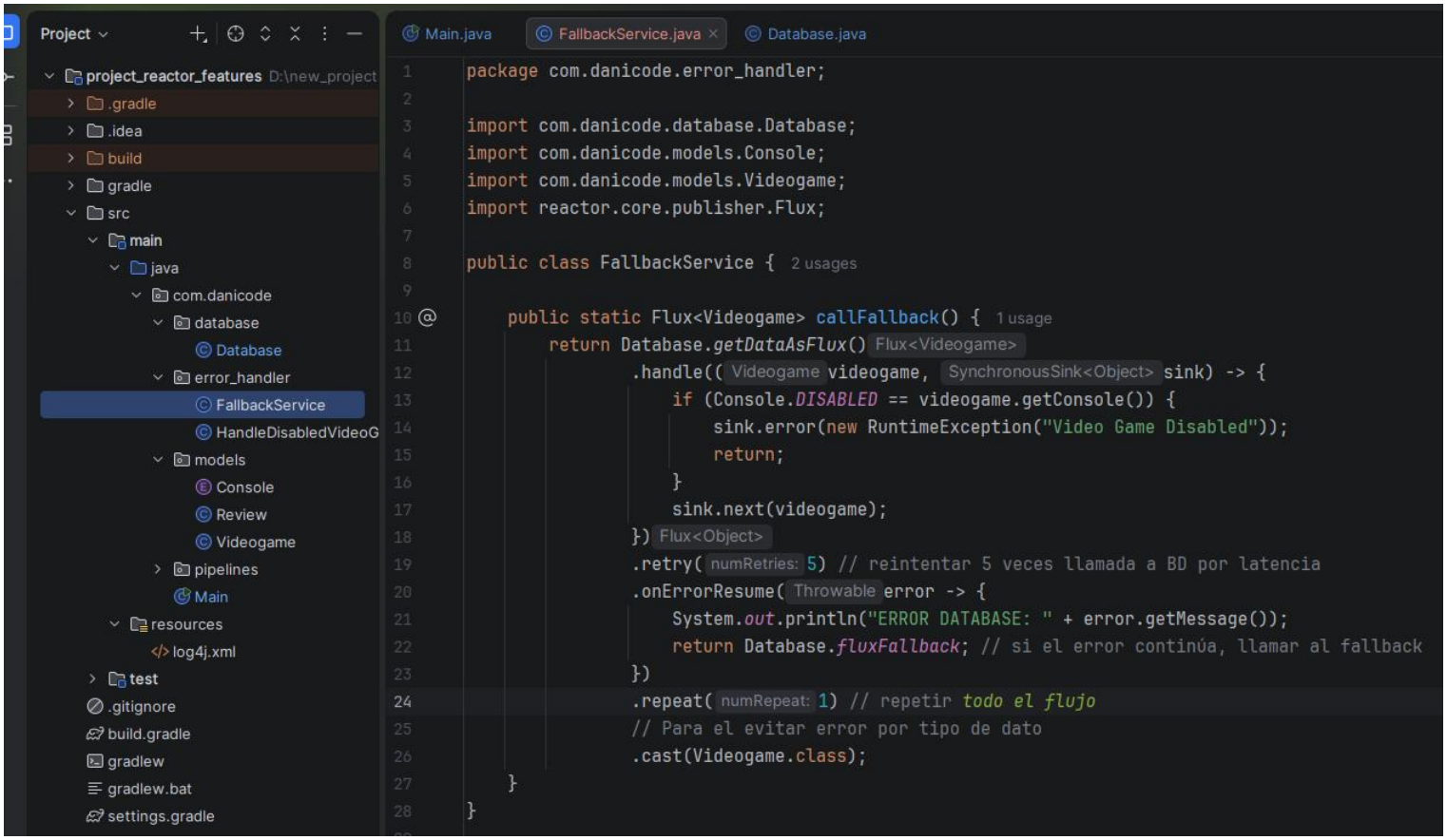
➔Se diferencia con el onErrorResume, es que este se usa cuando se tenga datos de respaldo. En cambio, el onErrorReturn regresa un único valor estático y por defecto: ejemplo, si se cae un micro y no se tiene datos de respaldo.



Main:

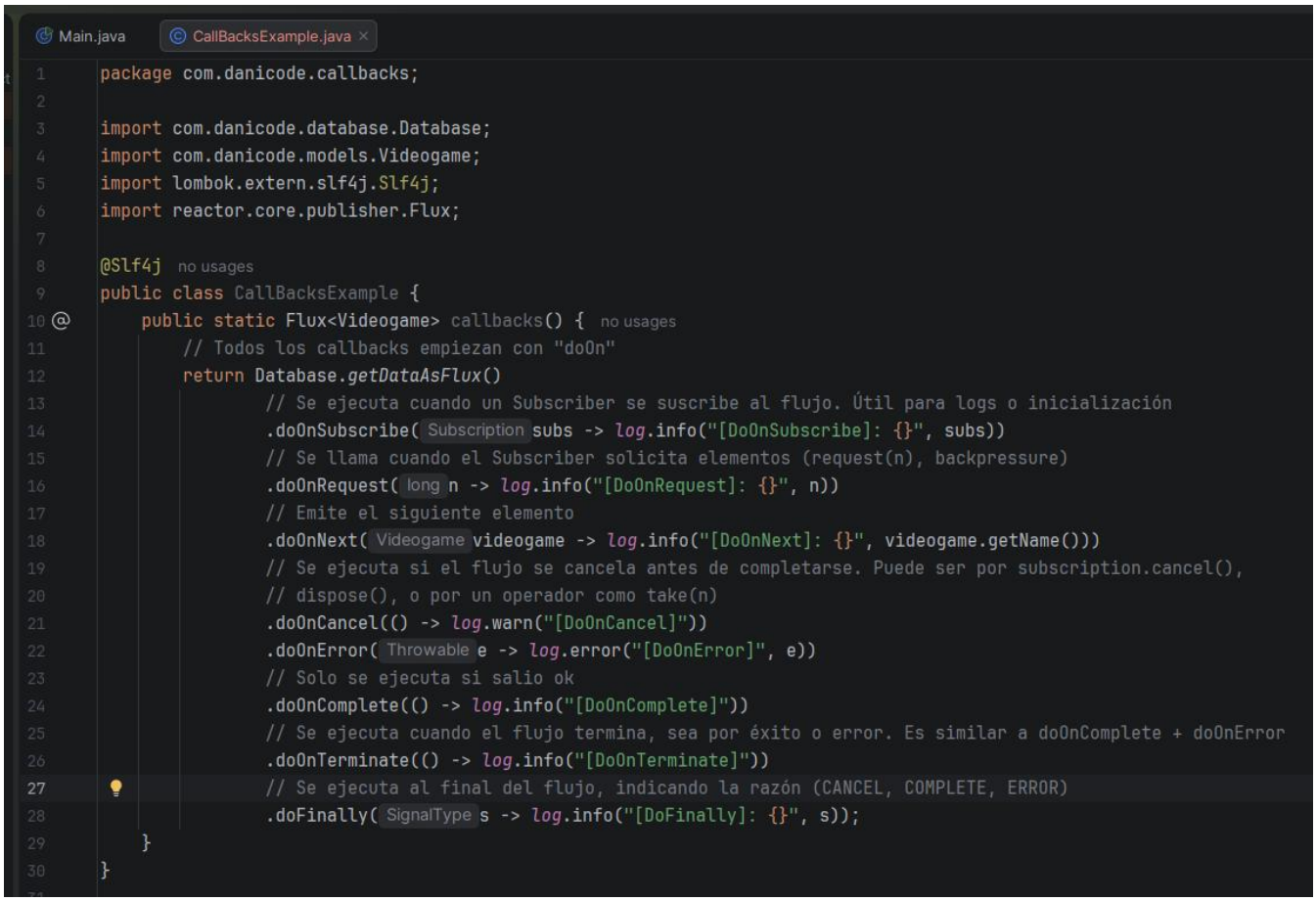
```
log.info( msg: "****** HANDLER ******");
HandleDisabledVideoGame.handleDisabledVideoGameDefault().subscribe(System.out::println );
```

26. Retry y Repeat



27. Callbacks – Parte 1

➔Función que se ejecuta en respuesta a un evento.



28. Callbacks – Parte 2

```
        .subscribe(System.out::println);*/
    System.out.println("CALLBACKS");
    CallBacksExample.callbacks().subscribe();
```

```
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
CALLBACKS
[DoOnSubscribe]: {}
[DoOnRequest]: 9223372036854775807
[DoOnNext]: Forza Horizon 5
[DoOnNext]: Resident Evil 4
[DoOnNext]: Assassin's Creed Origins
[DoOnNext]: Assassin's Creed Odyssey
[DoOnNext]: Assassin's Creed Valhalla
[DoOnNext]: GTA 5
[DoOnNext]: Crash Bandicoot 4
[DoOnNext]: Forza Horizon 4
[DoOnNext]: The Last of Us
[DoOnNext]: God of War
[DoOnNext]: Halo
[DoOnNext]: Super Mario Bros.
[DoOnNext]: Geometry Dash PC
[DoOnNext]: Luigi's Mansion
[DoOnNext]: Mario Party
[DoOnNext]: Call of Duty: Black Ops
[DoOnNext]: Call of Duty: Mobile
[DoOnNext]: Geometry Dash
[DoOnNext]: Candy Crush
[DoOnNext]: Forza Horizon 5 Edición Deluxe
```

```
[DoOnNext]: Cyberpunk 2077
[DoOnNext]: Fortnite
[DoOnNext]: Elden Ring
[DoOnNext]: FIFA 23
[DoOnNext]: Minecraft
[DoOnNext]: League of Legends
[DoOnNext]: Clash Royale
[DoOnComplete]
[DoOnTerminate]
[DoFinally]: onComplete

Process finished with exit code 0
```

```
    System.out.println("CALLBACKS");
    CallBacksExample.callbacks()
            .subscribe( Videogame data -> System.out.println(data.getName()),
                    Throwable error -> System.out.println(error.getMessage()),
                    () -> System.out.println("DONE - FINISH SUBS"));
```

29. Contexto (Context)

Operaciones basadas en el contexto.

```
private static boolean videoGameForConsole(Videogame videoGame, Console console) {  no usages  new *
    return videoGame.getConsole() == console || videoGame.getConsole() == Console.ALL;
}
```

```
        Database.getDataAsFlux()
                // filterWhen es async
                .filterWhen( Videogame game -> Mono.deferContextual( ContextView contextView -> {
                        var userId = contextView.getOrDefault( key: "userId",  defaultValue: "0");
                        if (userId.startsWith("1")) {
                            return Mono.just(videoGameForConsole(game, Console.XBOX));
                        } else if (userId.startsWith("2")) {
                            return Mono.just(videoGameForConsole(game, Console.PLAYSTATION));
                        }
                        return Mono.just( data: false);
                    }
                ))
                // Contexto antes del subcribe
                .contextWrite(Context.of("userId", "10020192"))
                .subscribe( Videogame game -> System.out.println("Recomendación: " + game.getName() + " Console: " + game.getConsole()));
    }

    private static boolean videoGameForConsole(Videogame videoGame, Console console) {  2 usages  new *
        return videoGame.getConsole() == console || videoGame.getConsole() == Console.ALL;
    }
}
```

30. Hot y Cold Publishers

```
// RECIBE LOS DATOS DESDE EL ORIGEN (DESDE EL CERO)
System.out.println("Cold Publishers");
Flux<Integer> coldPublishers = Flux.range( start: 1,  count: 10);
System.out.println("Subs 1 subscribed");
coldPublishers.subscribe( Integer n -> System.out.println("[S1]: " + n));
System.out.println("Subs 2 subscribed");
coldPublishers.subscribe( Integer n -> System.out.println("[S2]: " + n));
System.out.println("Subs 3 subscribed");
coldPublishers.subscribe( Integer n -> System.out.println("[S3]: " + n));

// RECIBE LOS DATOS DESDE QUE SE SUSCRIBE
System.out.println("Hot Publishers");
Flux<Long> hotPublishers = Flux.interval(Duration.ofSeconds(1)) Flux<Long>
        .publish() ConnectableFlux<Long>
        // Espera al menos un suscriptor para empezar a emitir datos
        .autoConnect();
System.out.println("Subs 4 subscribed");
hotPublishers.subscribe( Long n -> System.out.println("[S4]: " + n));
Thread.sleep( millis: 2000);
System.out.println("Subs 5 subscribed");
hotPublishers.subscribe( Long n -> System.out.println("[S5]: " + n));
Thread.sleep( millis: 1000);
System.out.println("Subs 6 subscribed");
hotPublishers.subscribe( Long n -> System.out.println("[S6]: " + n));
Thread.sleep( millis: 10000);
```